

Growth of Functions

1) How do we measure the efficiency of an algorithm and how do we measure the relative performances of alternative algorithms?

An algorithm solves a problem. We are interested in the time or space required to solve larger and larger instances of the problem.

Associated with a problem is a size (an integer) which is a measure of the quantity of input data.

→ The time required by an algorithm expressed as a function of the size of a problem is called the time complexity of an algorithm. The limiting behaviour of complexity as size increases is called the asymptotic time complexity (size of the input increases without bound).

In other words we look at input sizes large enough to make only the order of growth of the running time relevant.

Although we can ^{sometimes} determine the exact running time of an algorithm, the extra precision is usually not worth the effort of computing it.

For large enough inputs, the multiple constants and lower-order terms of an exact running time are dominated by the effects of the input size itself.

→ Usually, an algorithm that is asymptotically more efficient will be the best choice for all but very small inputs. An algorithm with a rapid growth rate might have a smaller constant of proportionality than one with a lower growth rate. In that case the rapidly growing algorithm might be superior for small problems, possibly even for all problems of a size that would interest us.

2) How does the asymptotic complexity determine the size of problems that can be solved by the algorithm?

We may be limited in the amount of time provided to solve a problem. Using the time complexity of an algorithm we can find out the time taken by the algorithm to solve problems of different size. Thus we can compute the maximum size of problem that can be solved using the algorithm.

One might suspect that the tremendous increase in the speed of calculations brought

about by the advent of the present generation of digital computers would decrease the importance of efficient algorithms. However, just the opposite is true. As computers become faster, we can handle larger problems. It is the complexity of an algorithm that determines the increase in problem size that can be achieved with an increase in computer speed.

3) Different Asymptotic notations:-

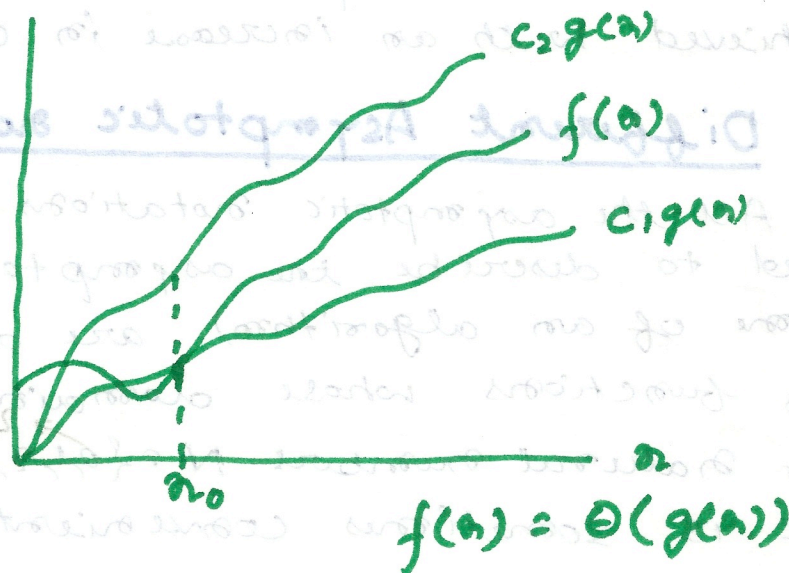
All the asymptotic notations (or notations used to describe the asymptotic running time of an algorithm) are defined in terms of functions whose domains are the set of natural numbers $N = \{0, 1, 2, \dots\}$. However, it is sometimes convenient to abuse the asymptotic notation in a variety of ways. For example, the notation can be easily extended to the domain of real numbers, or restricted to a subset of the natural numbers.

1) Θ notation:- For any function $g(n)$ $\Theta(g(n))$ is defined as:-

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0.\}$$

A function $f(n)$ is said to belong to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be sandwiched between $c_1g(n)$ and $c_2g(n)$, for sufficiently large n . We write $f(n) \in \Theta(g(n))$ or $f(n) = \Theta(g(n))$ to indicate that $f(n)$ is a member of $\Theta(g(n))$.

We say that $g(n)$ is an asymptotically tight bound for $f(n)$.



Note that, the definition of Θ notation requires $f(n)$ to be asymptotically non-negative (i.e. for sufficiently large values of n (i.e. $n \geq n_0$) $f(n)$ should be non-negative). Since, c_1, c_2 are +ve constants the ~~base~~ definition $0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$ requires $g(n)$ also to be asymptotically non-negative.

→ To determining the asymptotic tight bound for a function we ignore the lower-order terms and the coefficient of the

highest-order term.

For example, $\frac{1}{2}n^2 - 3n = \Theta(n^2)$

To find c_1, c_2 and n_0 :-

$$0 \leq cn^2 \leq \frac{1}{2}n^2 - 3n \leq c_2n^2 \quad \forall n \geq n_0$$

$$\Rightarrow c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2, \text{ which yields}$$

$$\Rightarrow \frac{6}{1-2c_1} \leq n \leq \frac{6}{1-2c_2}$$

$\therefore n$ is the interval $[n_0, \infty)$

$$\frac{6}{1-2c_2} \rightarrow \infty \Rightarrow 1-2c_2 = 0 \Rightarrow c_2 = \frac{1}{2}$$

We may choose any value for $\frac{6}{1-2c_1} \geq 0$

$$\text{Let } c_1 = \frac{1}{14}, \text{ then } \frac{6}{1-2c_1} = 7 \text{ and } 7 \leq n < \infty$$

$$\text{Thus, } n_0 = 7, c_1 = \frac{1}{14} \text{ and } c_2 = \frac{1}{2}$$

Since, some choice for n_0, c_1, c_2 exists $\frac{1}{2}n^2 - 3n = \Theta(n^2)$

$$\rightarrow f(n) = an^2 + bn + c = \Theta(n^2) \text{ where } a > 0.$$

Since any constant is a degree 0 polynomial

We can express any constant as $\Theta(n^0)$ or $\Theta(1)$.

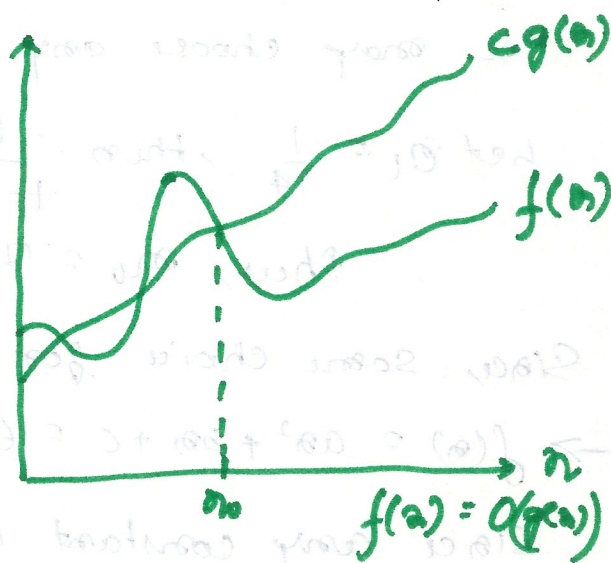
This notation is a minor abuse as it's not clear which variable is leading to infinity.

2) O-notation :-

When the Θ -notation asymptotically binds a function from above and below, we use O-notation ~~is~~ to simply specify an asymptotic upper bound. For any function $g(n)$,

$$O(g(n)) = \left\{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0 \right\}$$

We say $g(n)$ is an asymptotic upper bound for $f(n)$.



Note that $f(n) = \Theta(g(n))$ implies $f(n) = O(g(n))$ since Θ -notation is a stronger notation than O-notation.

$$\rightarrow an^2 + bn + c = \Theta(n^2) = O(n^2), a > 0$$

$\rightarrow$ When we say $f(n) = O(g(n))$, we merely claim that some constant multiple of $g(n)$ is an asymptotic upper bound on $f(n)$ with no claim about how tight an upper bound it is.

→ Since O -notation specifies an upper bound when we use it to bind the worst-case running time of an algorithm, we have a bound on the running time of the algorithm for every input size.

Thus an $O(n^2)$ bound on worst-case running time of insertion sort also applies to its running time on every input.

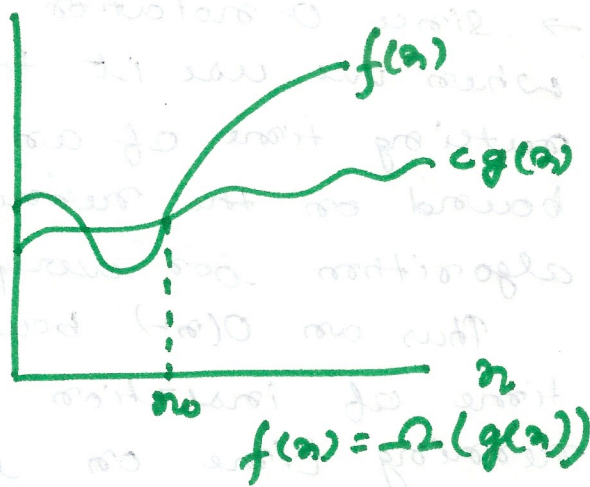
However the Θ -notation specifies a tight bound. Thus the bound may differ depending upon the nature of the input. For example, $\Theta(n^2)$ is the bound on the worst case running time of insertion sort, whereas for already sorted input insertion sort runs in $\Theta(n)$ time.

→ When we say that the running time of an algorithm is $O(n^2)$ we mean that the worst case running time is $O(n^2)$. Otherwise, technically it is an abuse to say the running time of an algorithm is $O(n^2)$.

3) Ω -notation:-

Big omega notation provides an asymptotic lower bound on a function. For any function $g(n)$

$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0 \}$



4) Theorem :-

For any 2 functions $f(n)$ and $g(n)$

we have $f(n) = \Theta(g(n))$ if and only if

$f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

→ When we use the Ω -notation to bound the best case running time of an algorithm, we implicitly bound the running time of the algorithm on arbitrary inputs.

When we say that the running time of insertion sort is $\Omega(n)$ we imply that the

best case running time of insertion sort is

$\Omega(n)$. Thus Big-Omega specifies an overall lower bound.